

05 · Deploy de IA em Produção: FastAPI, Docker e Cloud

□ 05 · Deploy de IA em Produção: FastAPI, Docker e Cloud

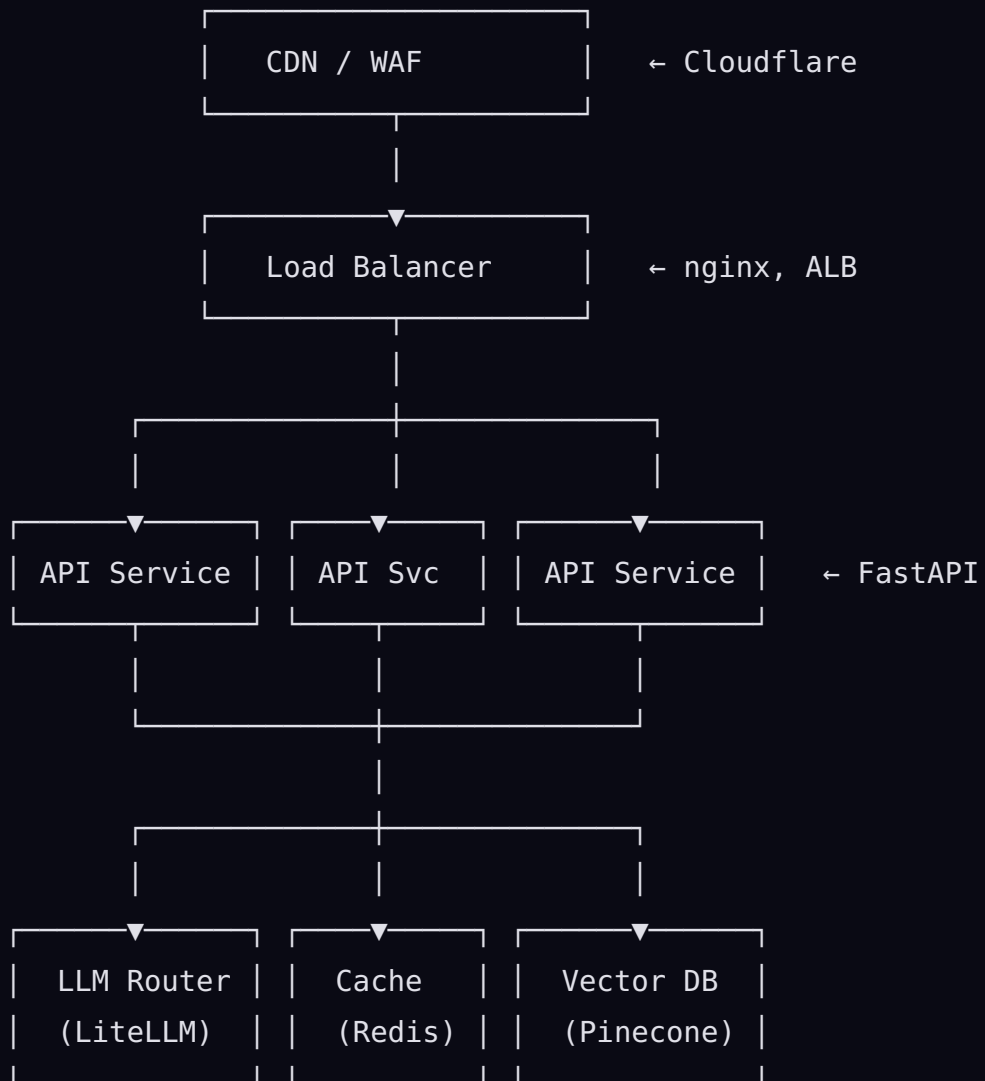
Tempo: 110 min · *Nível:* Master · *Pré-requisito:* 04 - RAG em Produção

Os 4 Estágios da Maturidade em Deploy de IA

1. **Notebook:** funciona pra demo
2. **API em servidor único:** 100 usuários
3. **Containerizado em cloud:** 10k usuários, auto-scaling
4. **Plataforma completa:** milhões, multi-região, SLOs, A/B testing

Este curso leva você do estágio 1 ao 3.

Arquitetura Recomendada



FastAPI: A Base



```

# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from openai import AsyncOpenAI
import time

app = FastAPI(title="Nexus AI Gateway", version="1.0.0")
client = AsyncOpenAI()

class CompletionRequest(BaseModel):
    prompt: str = Field(..., min_length=1, max_length=100_000)
    model: str = Field("gpt-4o-mini")
    temperature: float = Field(0.7, ge=0.0, le=2.0)

class CompletionResponse(BaseModel):
    text: str
    model: str
    tokens_in: int
    tokens_out: int
    latency_ms: int

@app.post("/v1/completions", response_model=CompletionResponse)
async def create_completion(req: CompletionRequest):
    start = time.perf_counter()
    try:
        response = await client.chat.completions.create(
            model=req.model,
            messages=[{"role": "user", "content": req.prompt}],
            temperature=req.temperature,
        )
        text = response.choices[0].message.content
        latency = int((time.perf_counter() - start) * 1000)

        return CompletionResponse(
            text=text,
            model=response.model,
            tokens_in=response.usage.prompt_tokens,

```

```
        tokens_out=response.usage.completion_tokens,
        latency_ms=latency,
    )
except Exception as e:
    logger.exception("completion_failed", extra={"error": str(e)})
    raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Caching Semântico: 50-80% de Economia

A maioria das perguntas se repete. Cache semântico detecta similaridade.

```

import hashlib
import numpy as np
import redis
import json

r = redis.Redis(host="redis", port=6379)

def embed(text: str) -> np.ndarray:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=[text]
    )
    return np.array(response.data[0].embedding)

def cache_key(embedding: np.ndarray) -> str | None:
    """Procura cache por similaridade de cosseno."""
    for key in r.scan_iter(match="cache:*"):
        cached_emb = np.frombuffer(r.hget(key, "emb"), dtype=np.float32)
        sim = float(embedding @ cached_emb / (
            np.linalg.norm(embedding) * np.linalg.norm(cached_emb)
        ))
        if sim >= 0.97:
            return key.decode()
    return None

def cached_completion(prompt: str, **kwargs) -> dict:
    emb = embed(prompt)
    key = cache_key(emb)
    if key:
        return json.loads(r.get(key))

    # Cache miss
    response = client.chat.completions.create(
        model=kwargs.get("model", "gpt-4o-mini"),
        messages=[{"role": "user", "content": prompt}],
    )
    result = {

```

```

        "text": response.choices[0].message.content,
        "tokens_in": response.usage.prompt_tokens,
        "tokens_out": response.usage.completion_tokens,
    }

    # Armazenar
    new_key = f"cache:{hashlib.md5(prompt.encode()).hexdigest()}"
    r.setex(new_key, 86400, json.dumps(result)) # TTL 24h
    r.hset(new_key, "emb", emb.astype(np.float32).tobytes())
    return result

```

LiteLLM: Multi-Provider Router

```

# litellm_config.yaml
model_list:
  - model_name: gpt-4o
    litellm_params:
      model: openai/gpt-4o
      api_key: os.environ/OPENAI_API_KEY

  - model_name: gpt-4o-mini
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: os.environ/OPENAI_API_KEY

  - model_name: claude-sonnet
    litellm_params:
      model: anthropic/claude-3-5-sonnet-20241022
      api_key: os.environ/ANTHROPIC_API_KEY

router_settings:
  num_retries: 3
  timeout: 30
  cooldown_time: 60

```

```
import litellm
litellm.load_config("litellm_config.yaml")

response = litellm.completion(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
)
```


Observabilidade: Langfuse

```
from langfuse import Langfuse
import os

langfuse = Langfuse(
    public_key=os.getenv("LANGFUSE_PUBLIC_KEY"),
    secret_key=os.getenv("LANGFUSE_SECRET_KEY"),
)

def traced_completion(user_id: str, prompt: str, model: str = "gpt-4o-mini"):
    trace = langfuse.trace(name="user_completion", user_id=user_id)
    generation = trace.generation(
        name="chat_completion",
        model=model,
        input=prompt,
        output=None, # preenchido depois
        metadata={"feature": "qa"},
    )

    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
    )
    text = response.choices[0].message.content

    generation.update(
        output=text,
        usage={
            "input": response.usage.prompt_tokens,
            "output": response.usage.completion_tokens,
        },
    )
    return text
```

Docker Multi-Stage

```
# Dockerfile
FROM python:3.12-slim AS builder
WORKDIR /build
RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen

FROM python:3.12-slim
WORKDIR /app
RUN useradd --create-home --shell /bin/bash appuser

COPY --from=builder /build/.venv /app/.venv
COPY . /app
ENV PATH="/app/.venv/bin:$PATH"

USER appuser
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Deploy: 3 Opções

Opção A: Fly.io (rápido)

```
curl -L https://fly.io/install.sh | sh
fly auth signup
fly launch --name nexus-ai-gateway
fly secrets set OPENAI_API_KEY=sk-proj-...
fly secrets set REDIS_URL=redis://...
fly deploy
```

Opção B: Railway (simples)

```
npm install -g @railway/cli  
railway login  
railway init  
railway variables set OPENAI_API_KEY=sk-proj-...  
railway up
```

Opção C: Kubernetes (escala)

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus-ai-gateway
spec:
  replicas: 3
  selector:
    matchLabels: {app: ai-gateway}
  template:
    metadata:
      labels: {app: ai-gateway}
    spec:
      containers:
        - name: api
          image: nexus/ai-gateway:1.0.0
          resources:
            requests: {memory: "512Mi", cpu: "500m"}
            limits: {memory: "1Gi", cpu: "1000m"}
          ports:
            - containerPort: 8000
          envFrom:
            - secretRef:
                name: nexus-secrets
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ai-gateway-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nexus-ai-gateway
  minReplicas: 3
  maxReplicas: 20
  metrics:
```

```
- type: Resource
  resource:
    name: cpu
    target: {type: Utilization, averageUtilization: 70}
```

SLOs Obrigatórios

SLO	Target
Disponibilidade	> 99.5%
Latência p95	< 3s
Latência p99	< 8s
Taxa de erro	< 1%
Custo/1k requests	< \$2

Alertas Prometheus

```
groups:
- name: ai-gateway
  rules:
  - alert: HighErrorRate
    expr: |
      sum(rate(api_errors_total[5m])) /
      sum(rate(api_requests_total[5m])) > 0.05
    for: 5m
  - alert: HighLatency
    expr: |
      histogram_quantile(0.95, rate(api_re-
        quest_duration_seconds_bucket[5m])) > 5
    for: 10m
```

Custos Típicos (10k requests/dia)

Componente	Custo/mês
FastAPI (Fly.io 3 pods)	\$15
Redis (Upstash)	\$10
Pinecone Serverless	\$70
OpenAI API	\$500-2000
Langfuse Cloud (free)	\$0
Total	\$600-2100/mês

Checklist de Produção

- ☐ API FastAPI assíncrona
- ☐ Cache semântico habilitado
- ☐ LiteLLM configurado
- ☐ Langfuse em produção
- ☐ Logs estruturados (JSON)
- ☐ SLOs definidos
- ☐ Alertas Prometheus ativos
- ☐ Rate limiting
- ☐ CI/CD automatizado
- ☐ Runbook de incidentes

Próximos Passos

- **Segurança:** curso 06 - Segurança e Jailbreaks
- **Disaster recovery:** playbook PB-DEPLOY

Recursos

- FastAPI: <https://fastapi.tiangolo.com>
- Fly.io: <https://fly.io/docs>
- LiteLLM: <https://docs.litellm.ai>
- Langfuse: <https://langfuse.com/docs>